LAB MANUAL

COM 682

Faculty of Computing, Engineering and The Built Environment

COM682 – Lab 04-a

Azure Storage Services

Contents

1. Azure Storage services

The Azure Storage platform is Microsoft's cloud storage solution for modern data storage scenarios. Core storage services offer a massively scalable object store for data objects, disk storage for Azure virtual machines (VMs), a file system service for the cloud, a messaging store for reliable messaging, and a NoSQL store. The services are:

- **Durable and highly available**. Redundancy ensures that your data is safe in the event of transient hardware failures. You can also opt to replicate data across data centres or geographical regions for additional protection from local catastrophes or natural disasters. Data replicated in this way remains highly available in the event of an unexpected outage.
- **Secure**. The service encrypts all data written to an Azure storage account. Azure Storage provides you with fine-grained control over who has access to your data.
- **Scalable**. Azure Storage is designed to be massively scalable to meet today's applications' data storage and performance needs.
- **Managed**. Azure handles hardware maintenance, updates, and critical issues for you.
- **Accessible**. Data in Azure Storage is accessible from anywhere in the world over HTTP or HTTPS. Microsoft provides client libraries for Azure Storage in various languages, including .NET, Java, Node.js, Python, PHP, Ruby, Go, and others, as well as a mature REST API. Azure Storage supports scripting in Azure PowerShell or Azure CLI. And the Azure portal and Azure Storage Explorer offer easy visual solutions for working with your data.

2. Core storage services

The Azure Storage platform includes the following data services:

- Azure Blobs: A massively scalable object store for text and binary data. It also includes support for big data analytics through Data Lake Storage Gen2.
- Azure Files: Managed file shares for cloud or on-premises deployments.
- Azure Queues: A messaging store for reliable messaging between application components.
- Azure Tables: A NoSQL store for schemaless storage of structured data.
- Azure Disks: Block-level storage volumes for Azure VMs.

3. Example scenarios

The following table compares Files, Blobs, Disks, Queues, and Tables and shows example scenarios for each.

Feature	Description	When to use
Azure Files	Offers fully managed cloud file shares that you can access from anywhere via the industry-standard Server Message Block (SMB) protocol. You can mount Azure file shares from the cloud or on-premises deployments of Windows, Linux, and macOS.	You want to "lift and shift" an application to the cloud that already uses the native file system APIs to share data between it and other applications running in Azure. You want to replace or supplement on-premises file servers or NAS devices. You want to store development and debugging tools that need to be accessed from many virtual machines.
Azure Blobs	Allows unstructured data to be stored and accessed at a massive scale in block blobs. It also supports Azure Data Lake Storage Gen2 for enterprise big data analytics solutions.	You want your application to support streaming and random access scenarios. You want to be able to access application data from anywhere. You want to build an enterprise data lake on Azure and perform big data analytics.
Azure Disks	Allows data to be persistently stored and accessed from an attached virtual hard disk.	You want to "lift and shift" applications that use native file system APIs to read and write data to persistent disks. You want to store data that is not required to be accessed from outside the virtual machine to which the disk is attached.

Azure Queues	Allows for asynchronous message queueing between application components.	You want to decouple application components and use asynchronous messaging to communicate between them. For guidance around when to use Queue storage versus Service Bus queues, see Storage queues and Service Bus queues - compared and contrasted.
Azure Tables	Allow you to store structured NoSQL data in the cloud, providing a key/attribute store with a schemaless design.	You want to store flexible datasets like user data for web applications, address books, device information, or other types of metadata your service requires. For guidance around when to use Table storage versus the Azure Cosmos DB Table API, see Developing with Azure Cosmos DB Table API and Azure Table storage.

Blob storage

Azure Blob storage is Microsoft's object storage solution for the cloud. Blob storage is optimized for storing massive amounts of unstructured data, such as text or binary data.

Blob storage is ideal for:

- Serving images or documents directly to a browser.
- Storing files for distributed access.
- Streaming video and audio.
- Storing data for backup and restore disaster recovery and archiving.
- Storing data for analysis by an on-premises or Azure-hosted service.

Objects in Blob storage can be accessed from anywhere in the world via HTTP or HTTPS. Users or client applications can access blobs via URLs, the Azure Storage REST API, Azure PowerShell, Azure CLI, or an Azure Storage client library. The storage client libraries are available for multiple languages, including .NET, Java, Node.js, Python, PHP, and Ruby.

Azure Files

Azure Files enables you to set up highly available network file shares that can be accessed by using the standard Server Message Block (SMB) protocol. That means that multiple VMs can share the duplicate files with both read and write access. You can also read the files using the REST interface or the storage client libraries.

One thing that distinguishes Azure Files from files on a corporate file share is that you can access the files from anywhere in the world using a URL that points to the file and includes a shared access signature (SAS) token. You can generate SAS tokens; they allow specific access to a private asset for a specific amount of time.

File shares can be used for many common scenarios:

- Many on-premises applications use file shares. This feature makes it easier to migrate those applications that share data to Azure. If you mount the file share to the same drive letter that the on-premises application uses, the part of your application that accesses the file share should work with minimal, if any, changes.
- Configuration files can be stored on a file share and accessed from multiple VMs. Tools and utilities used by multiple developers in a group can be stored on a file share, ensuring that everybody can find them and that they use the same version.
- Resource logs, metrics, and crash dumps are just three examples of data that can be written to a file share and processed or analyzed later.

Queue storage

The Azure Queue service is used to store and retrieve messages. Queue messages can be up to 64 KB in size, and a queue can contain millions of messages. Queues are generally used to store lists of messages to be processed asynchronously.

For example, say you want your customers to be able to upload pictures, and you want to create thumbnails for each picture. You could have your customer wait for you to create the thumbnails while uploading the pictures. An alternative would be to use a queue. When the customer finishes their upload, write a message to the queue. Then have an Azure Function retrieve the message from the queue and create the thumbnails. Each of the parts of this processing can be scaled separately, giving you more control when tuning it for your usage.

Table storage

Azure Table storage is now part of Azure Cosmos DB. To see Azure Table storage documentation, see the Azure Table Storage Overview. In addition to the existing Azure Table storage service, there is a new Azure Cosmos DB Table API offering that provides throughput-optimized tables, global distribution, and automatic secondary indexes.

Disk storage

An Azure managed disk is a virtual hard disk (VHD). You can think of it like a physical disk in an on-premises server but, virtualized. Azure-managed disks are stored as page blobs, which are a random IO storage object in Azure. We call a managed disk 'managed' because it is an abstraction over page blobs, blob containers, and Azure storage accounts. With managed disks, all you have to do is provision the disk, and Azure takes care of the rest.

4. Storage Account.

An Azure storage account contains all of your Azure Storage data objects: blobs, file shares, queues, tables, and disks. The storage account provides a unique namespace for your Azure

Storage data that's accessible from anywhere in the world over HTTP or HTTPS. Data in your storage account is durable and highly available, secure, and massively scalable.

Types of Storage Account

Azure Storage offers several types of storage accounts. Each type supports different features and has its own pricing model. Consider these differences before you create a storage account to determine the type of account that's best for your applications.

The following table describes the types of storage accounts recommended by Microsoft for most scenarios. All of these use the Azure Resource Manager deployment model.

Type of storage account	Supported storage services	Redundancy options	Usage
Standard general-purpose v2	Blob (including Data Lake Storage[1]), Queue, and Table storage, Azure Files	LRS/GRS/RA-GRS ZRS/GZRS/RA-GZRS[2]	Standard storage account type for blobs, file shares, queues, and tables. Recommended for most scenarios using Azure Storage. Note that if you want support for NFS file shares in Azure Files, use the premium file shares account type.
Premium block blobs[3]	Blob storage (including Data Lake Storage[1])	LRS ZRS[2]	Premium storage account type for block blobs and append blobs. Recommended for scenarios with high transactions rates, or scenarios that use smaller objects or require consistently low storage latency. Learn more about example workloads.
Premium file shares[3]	Azure Files	LRS ZRS[2]	Premium storage account type for file shares only. Recommended for enterprise or high-performance scale

			applications. Use this account type if you want a storage account that supports both SMB and NFS file shares.
Premium page blobs[3]	Page blobs only	LRS	Premium storage account type for page blobs only. Learn more about page blobs and sample use cases.

[1] Data Lake Storage is a set of capabilities dedicated to big data analytics, built on Azure Blob storage. For more information, see Introduction to Data Lake Storage Gen2 and Create a storage account to use with Data Lake Storage Gen2.

[2] Zone-redundant storage (ZRS) and geo-zone-redundant storage (GZRS/RA-GZRS) are available only for standard general-purpose v2, premium block blobs, and premium file shares accounts in certain regions. For more information, see Azure Storage redundancy.

[3] Storage accounts in a premium performance tier use solid-state drives (SSDs) for low latency and high throughput.

Storage account endpoints

A storage account provides a unique namespace in Azure for your data. Every object that you store in Azure Storage has an address that includes your unique account name. The combination of the account name and the Azure Storage service endpoint forms the endpoints for your storage account.

When naming your storage account, keep these rules in mind:

- Storage account names must be between 3 and 24 characters in length and may contain numbers and lowercase letters only.
- Your storage account name must be unique within Azure. No two storage accounts can have the same name.

The following table lists the format of the endpoint for each of the Azure Storage services.

Storage service	**Endpoint**
Blob storage	https://<storage-account>.blob.core.windows.net
Data Lake Storage Gen2	https://<storage-account>.dfs.core.windows.net
Azure Files	https://<storage-account>.file.core.windows.net
Queue storage	https://<storage-account>.queue.core.windows.net
Table storage	https://<storage-account>.table.core.windows.net

Construct the URL for accessing an object in a storage account by appending the object's location in the storage account to the endpoint. For example, the URL for a blob will be similar to:

http://*mystorageaccount*.blob.core.windows.net/*mycontainer*/*myblob*

5. Migrate a storage account

This section focuses on storage migrations to Azure and provides guidance on the following storage migration scenarios:

- Migration of unstructured data, such as files and objects
- Migration of block-based devices, such as disks and storage area networks (SANs)

Migration of unstructured data

Migration of unstructured data includes following scenarios:

- File migration from network attached storage (NAS) to one of the Azure file offerings:
    - Azure Files
    - Azure NetApp Files
    - independent software vendor (ISV) solutions.
- Object migration from object storage solutions to the Azure object storage platform:
    - Azure Blob Storage
    - Azure Data Lake Storage

Migration phases

A full migration consists of several different phases: discovery, assessment, and migration.

Discovery	Assessment	Migration
- Discover sources to be migrated	- Assess applicable target service - Technical vs. cost considerations	- Initial migration - Resync - Final switch over

Discovery phase

In the discovery phase, you determine all sources that need to be migrated like SMB shares, NFS exports, or object namespaces. You can do this phase manually, or use automated tools.

Assessment phase

The assessment phase is critical in understanding available options for the migration. To reduce the risk during migration, and to avoid common pitfalls follow these three steps:

Assessment phase steps	Options
Choose a target storage service	- Azure Blob Storage and Data Lake Storage - Azure Files - Azure NetApp Files - ISV solutions
Select a migration method	- Online - Offline - Combination of both
Choose the best migration tool for the job	- Commercial tools (Azure and ISV) - Open source

Choose a target storage service

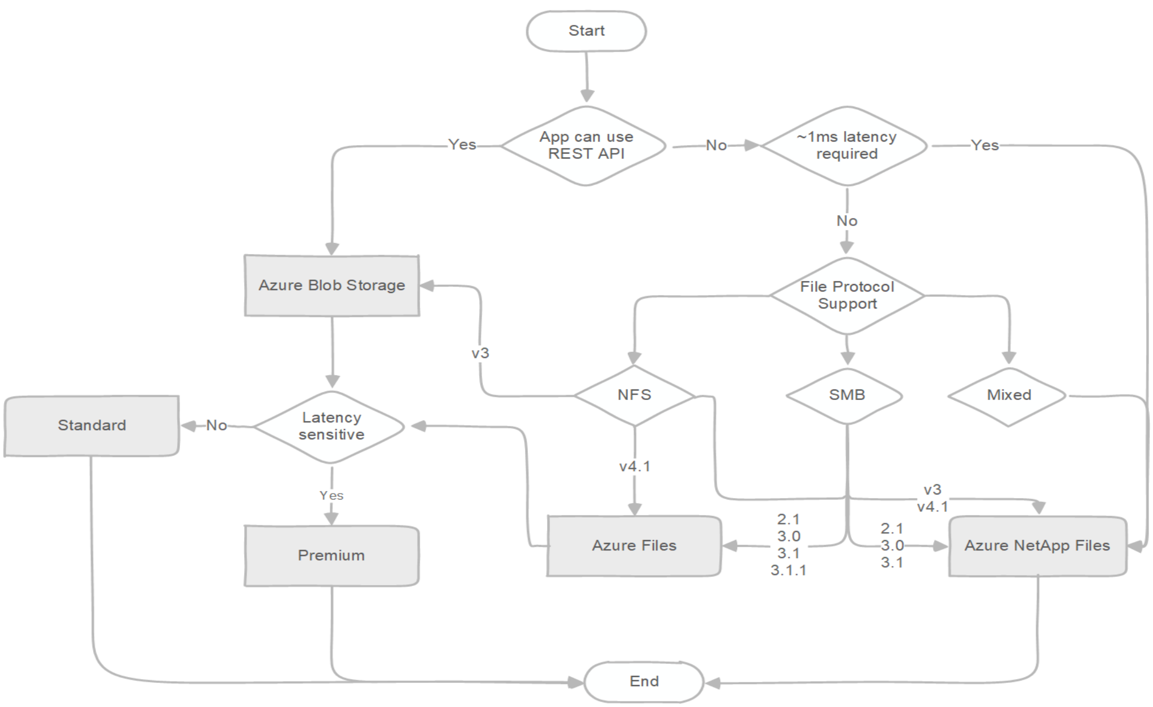
Choosing a target storage service depends on the application or users who access the data. The correct choice depends on both technical and financial aspects. First, do a technical assessment to assess possible targets and determine which services satisfy the requirements. Next, do a financial assessment to determine the best choice.

To help select the target storage service for the migration, evaluate the following aspects of each service:

- Protocol support
- Performance characteristics
- Limits of the target storage service

The following diagram is a simplified decision tree that helps guide you to the recommended Azure file service. If native Azure services do not satisfy requirements, a variety of [independent software vendor (ISV) solutions]() will.

After you finish the technical assessment, and select the proper target, do a cost assessment to determine the most cost-effective option.



To keep the decision tree simple, limits of the target storage service aren't incorporated in the diagram. To find out more about current limits, and to determine whether you need to modify your choices based on them, see:

- [Storage account limits]()

- Blob Storage limits
- Azure Files scalability and performance targets
- Azure NetApp Files resource limits

If any of the limits pose a blocker for using a service, Azure supports several storage vendors that offer their solutions on Azure Marketplace. For information about validated ISV partners that provide file services, see Azure Storage partners for primary and secondary storage.

Select the migration method

There are two basic migration methods for storage migrations.

- **Online**. The online method uses the network for data migration. Either the public internet or Azure ExpressRoute can be used. If the service doesn't have a public endpoint, you must use a VPN with public internet.
- **Offline.** The offline method uses one of the Azure Data Box devices.

The decision to use an online method versus an offline method depends on the available network bandwidth. The online method is preferred in cases where there's sufficient network bandwidth to perform a migration within the needed timeline.

It's possible to use a combination of both methods, offline method for the initial bulk migration and an online method for incremental migration of changes. Using both methods simultaneously requires a high level of coordination and isn't recommended for this reason. If you choose to use both methods isolate the data sets that are migrated online from the data sets that are migrated offline.

Choose the best migration tool for the job

There are various migration tools that you can use to perform the migration. Some are open source like AzCopy, robocopy, xcopy, and rsync while others are commercial. List of available commercial tools and comparison between them is available on our comparison matrix.

Open-source tools are well suited for small-scale migrations. For migration from Windows file servers to Azure Files, Microsoft recommends starting with Azure Files native capability and using Azure File Sync. For more complex migrations consisting of different sources, large capacity, or special requirements like throttling or detailed reporting with audit capabilities, commercial tools are the best choice. These tools make the migration easier and reduce the risk significantly. Most commercial tools can also perform the discovery, which provides a valuable input for the assessment.

Migration phase

Migration of block-based devices is typically done as part of virtual machine or physical host migration. It's a common misconception to delay block storage decisions until after the migration. Making these decisions ahead of time with appropriate considerations for workload requirements leads to a smoother migration to the cloud.

To explore workloads to migrate and approach to take, see the Azure Disk Storage documentation, and resources on the Disk Storage product page. You can learn about which disks fit your requirements, and the latest capabilities such as disk bursting. Migration of block based devices can be done in two ways:

- For migration of full virtual machines together with the underlying block-based devices, see the Azure Migrate documentation
- For migration of block based devices only, and more complexed use cases, use Cirrus Migrate Cloud.

6. Creating a Storage Account.

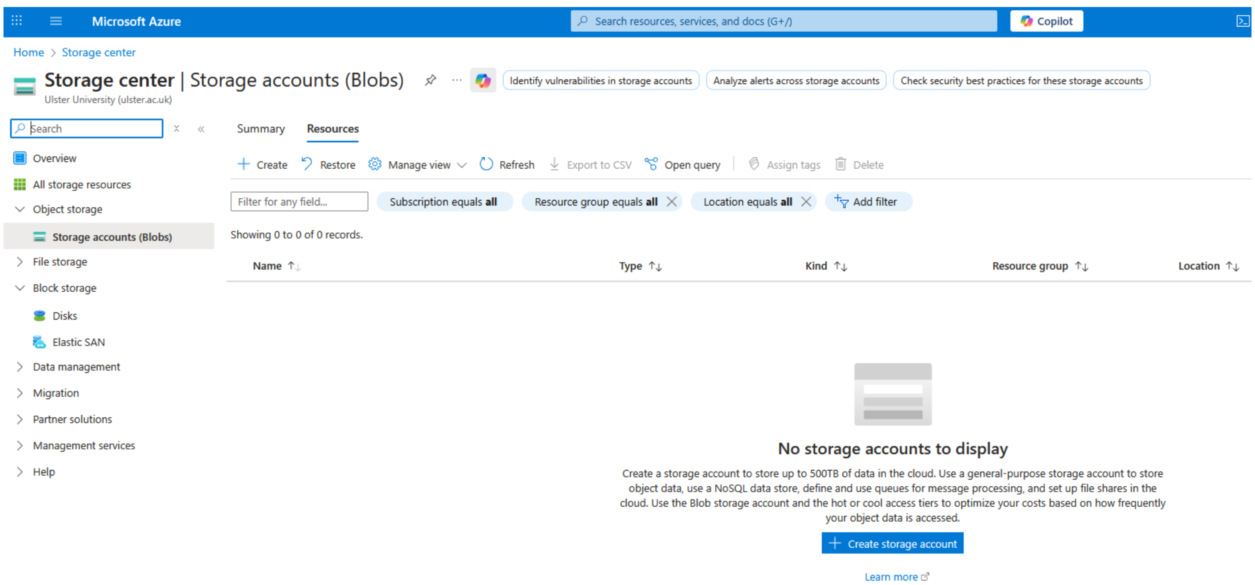
Open Azure portal : https://portal.azure.com/

Create a storage account

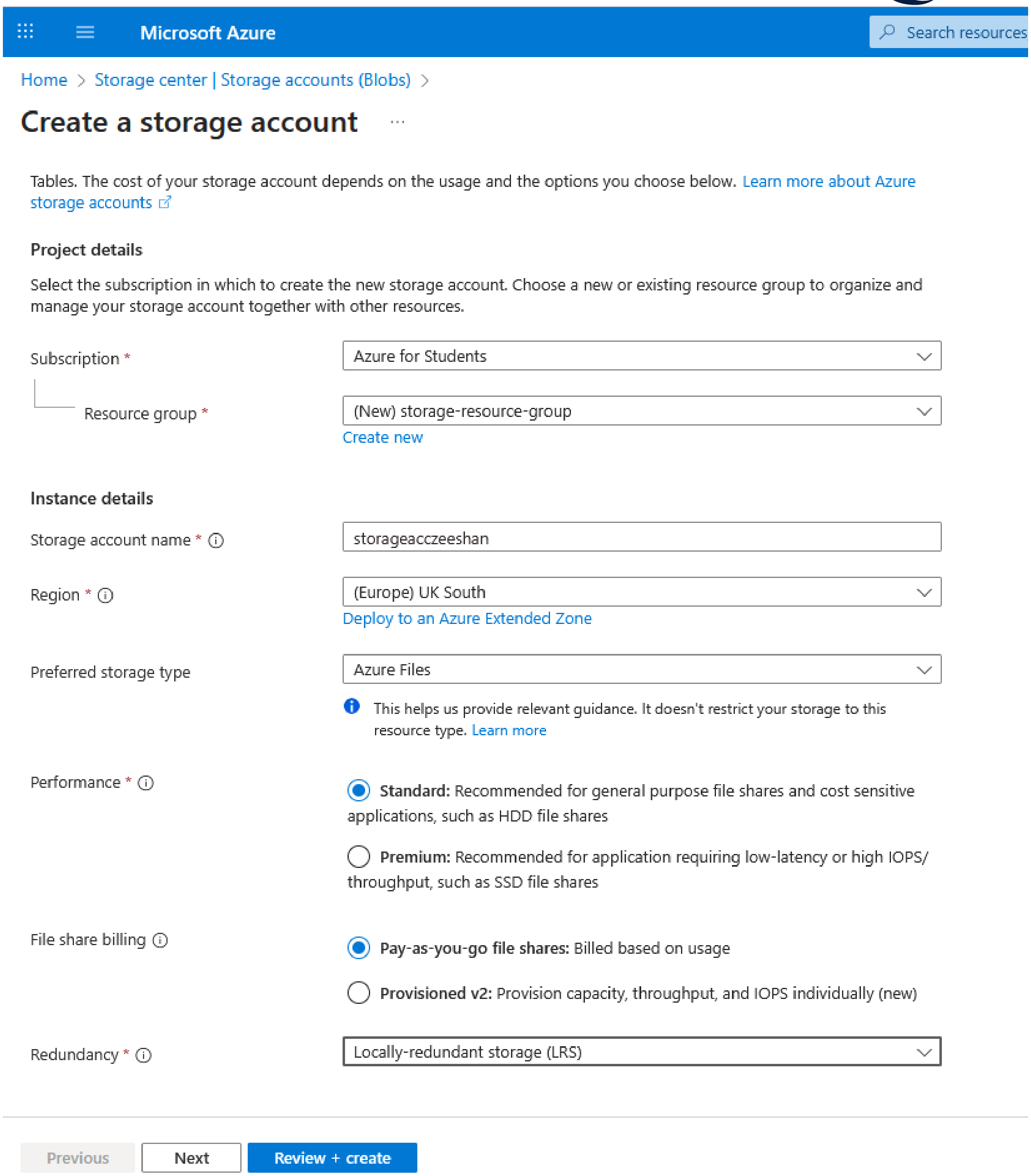
A storage account is an Azure Resource Manager resource. Resource Manager is the deployment and management service for Azure. For more information, see Azure Resource Manager overview.

Every Resource Manager resource, including an Azure storage account, must belong to an Azure resource group. A resource group is a logical container for grouping your Azure services. When you create a storage account, you have the option to either create a new resource group, or use an existing resource group. This how-to shows how to create a new resource group.

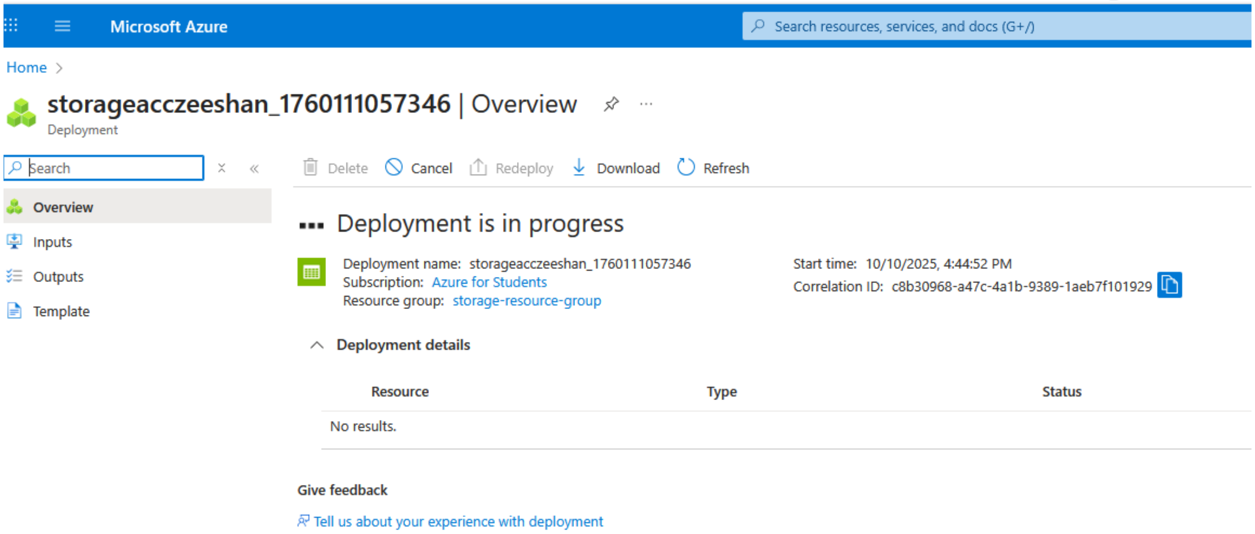
6.1.Search for Storage account in search bar



6.2.Click on +create storage account



6.3. Click Review + create



7. Linking static website to storage account.

Before you start with this section you will need simple html page to upload.

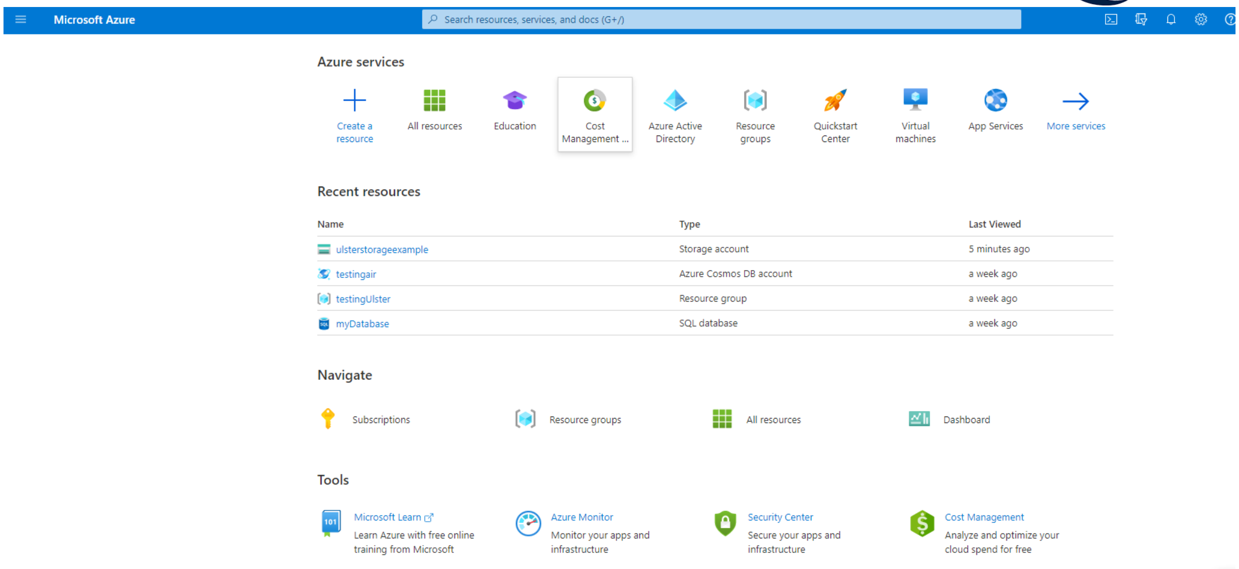
You can create a simple html page in notepad.

1. Open notepad and paste the following html code into it.

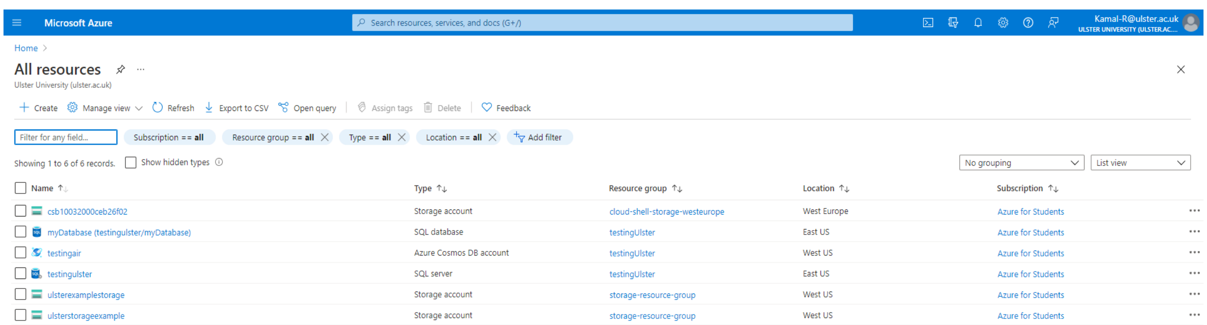
```
<html>
  <h1> Cloud Computing Lab </h1>
  <h2> Your Name </h2>
</html>
```

2. Please save the file as **index.html**
3. Open Azure portal.

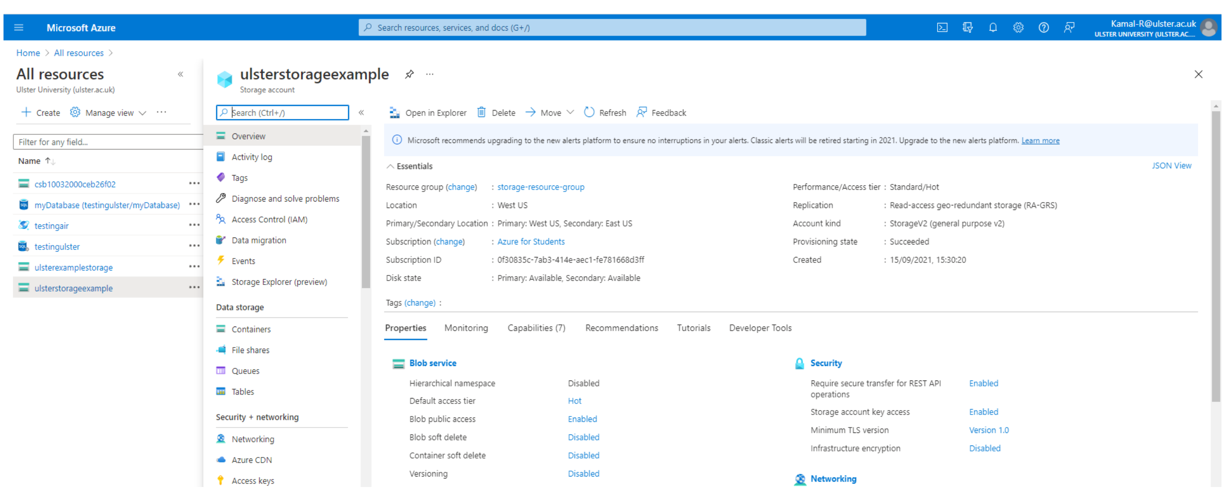
https://portal.azure.com/#home



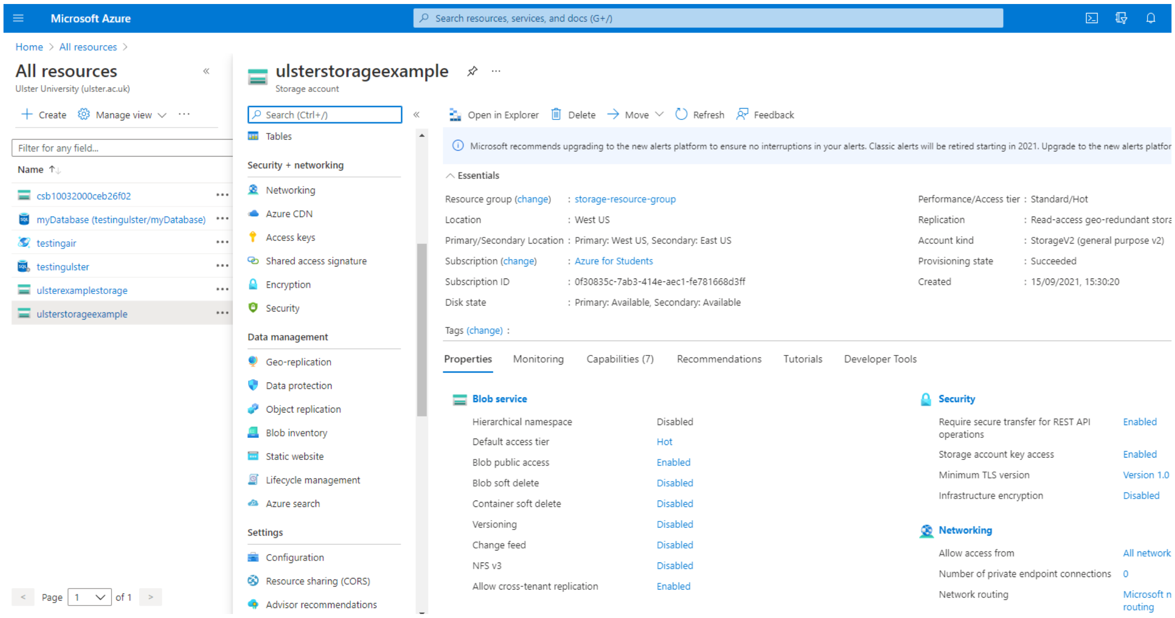
4. Click on All resources.



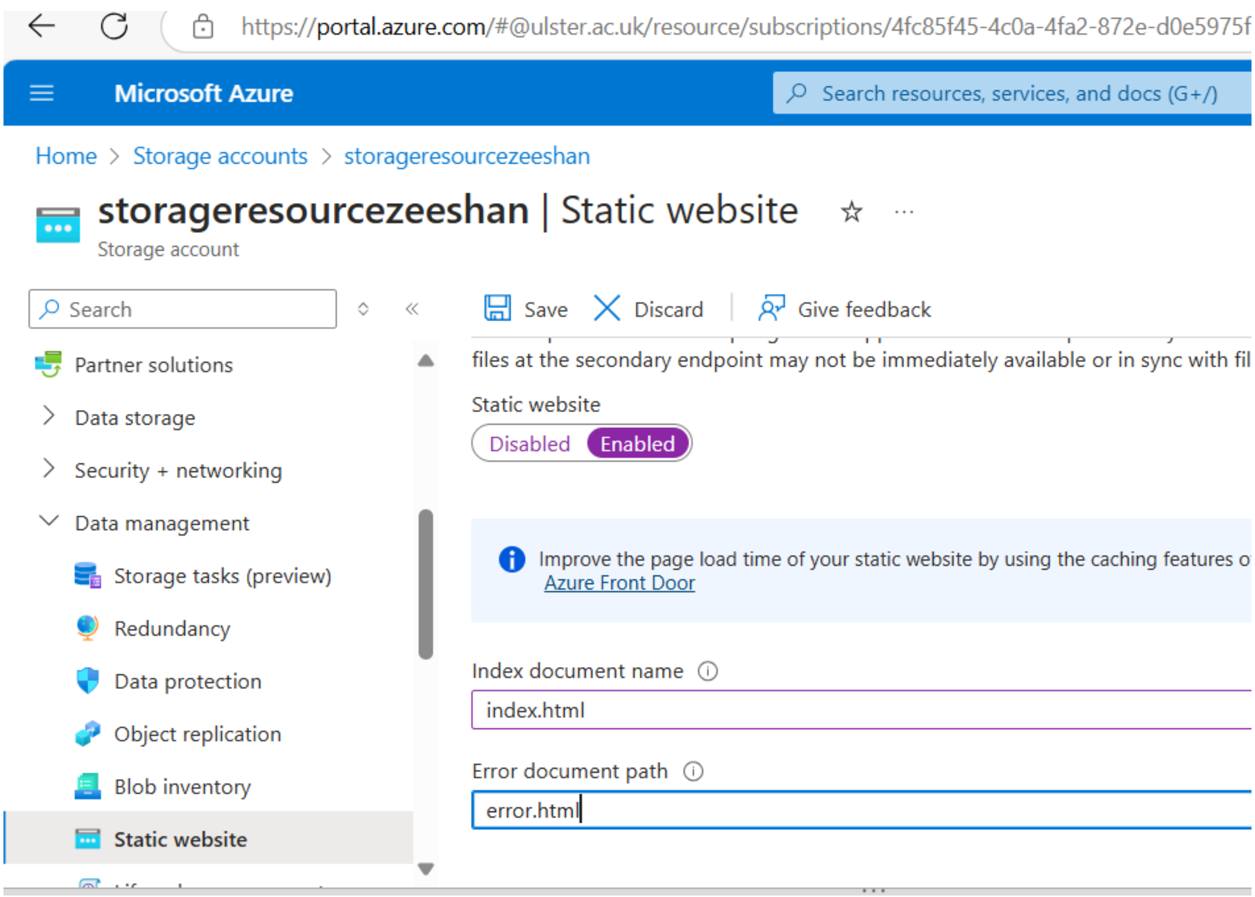
You should be able to see the storage account you have created in last section.



5. Drop down to data management.

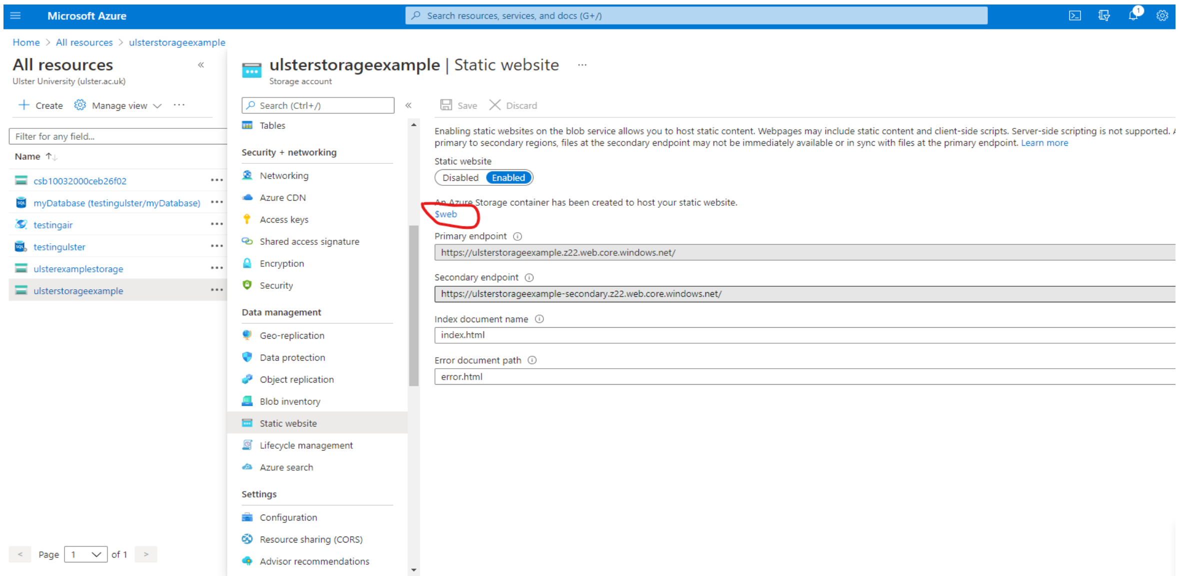


6. Click on static website

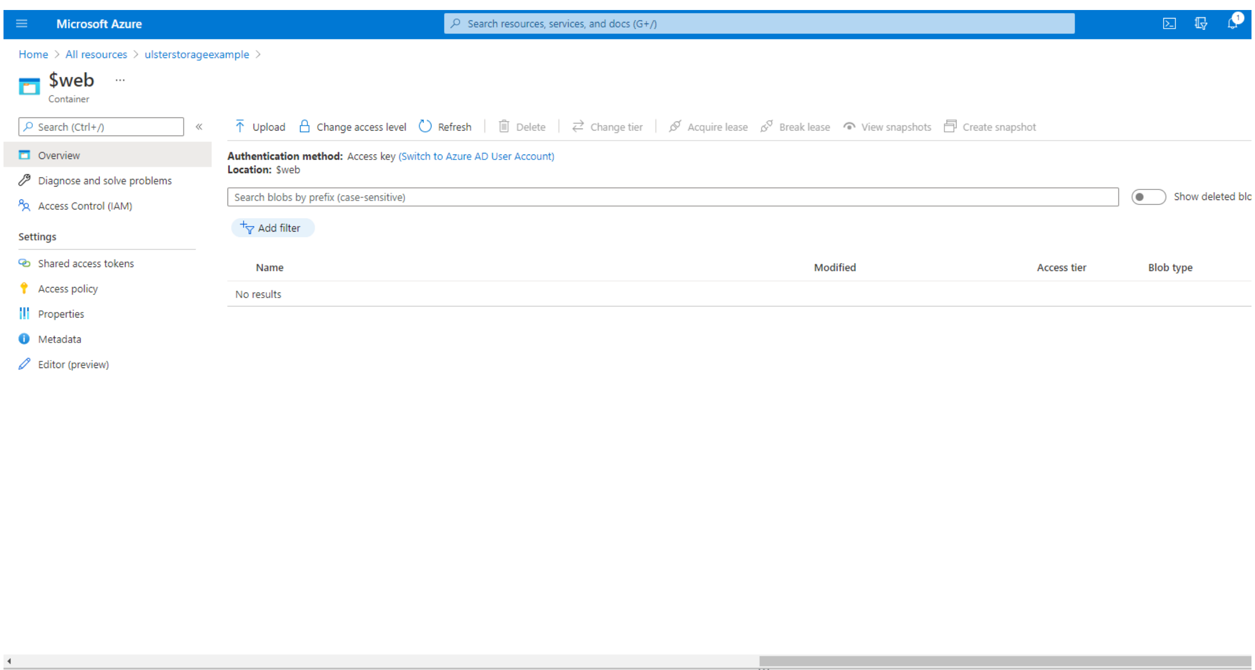


Click on Save

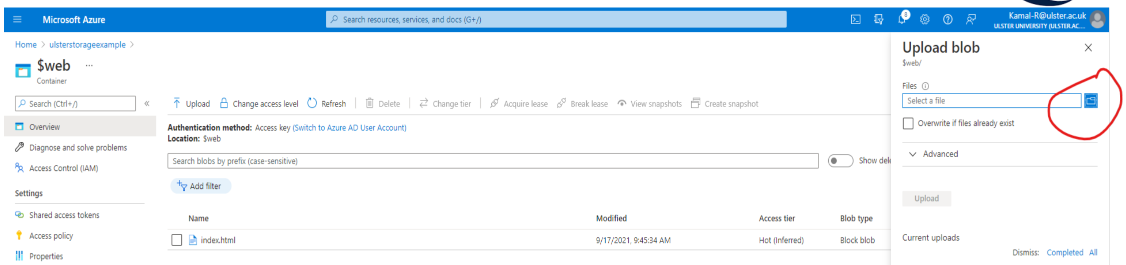
7. Now click on $web.



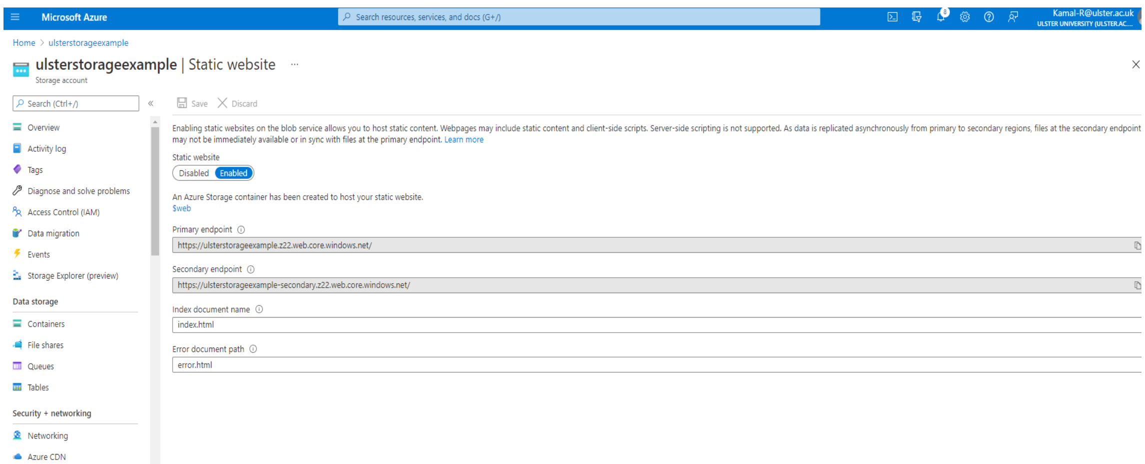
8. Please click on upload.



9. Upload blob panel should be visible now click on file icon and locate your **index.html** file which you have created earlier.



10. Get back to your static website page.



Copy the primary endpoint and paste it in the web browser.

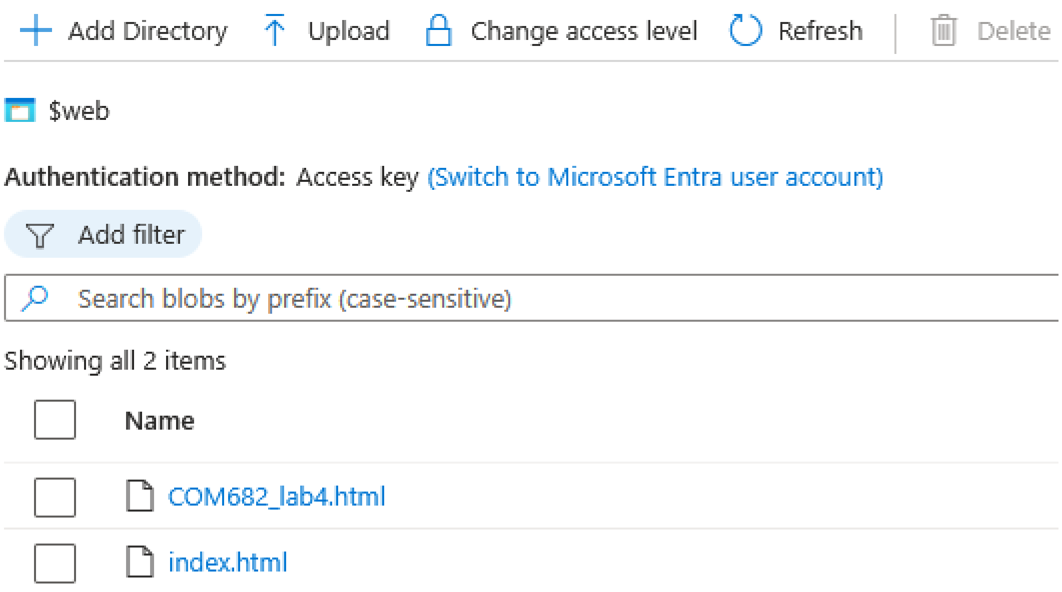
11. Go to primary end point.

You should be able to see your webpage now.

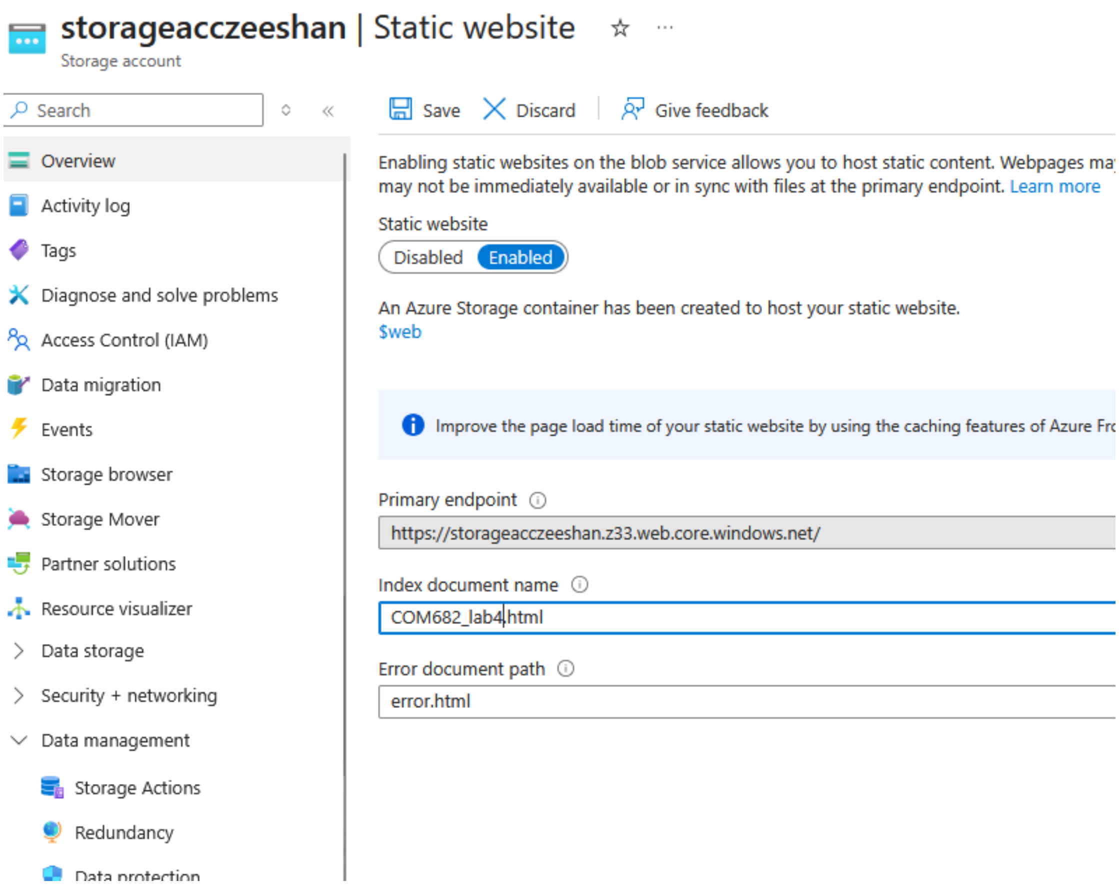
Note: you can upload a complete static website in blob and the process will be the same.

12. Now download the file ‘COM682_lab4.html’ from BB.

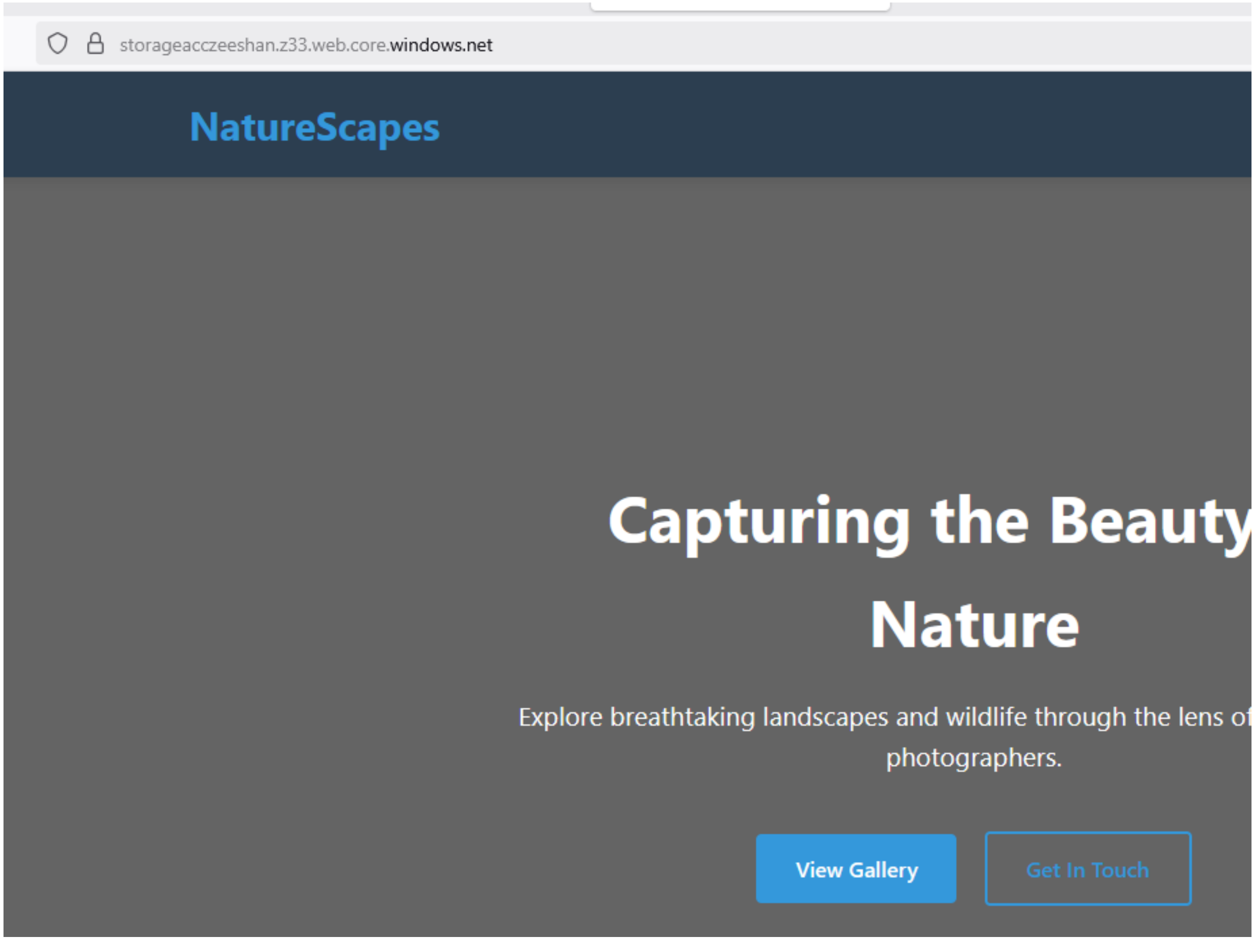
13. Upload the new html file at $web



14. Change the index document name to COM682_lab4.html.



Use Primary endpoint in different tab to access the website



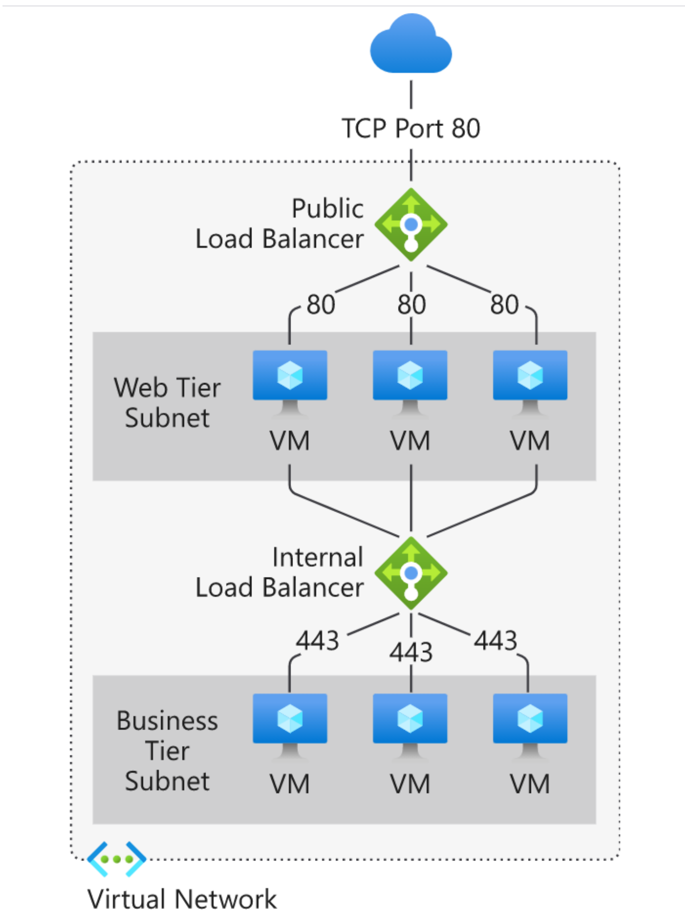
8. Azure Load Balancer

Load balancing refers to evenly distributing load (incoming network traffic) across a group of backend resources or servers.

Azure Load Balancer operates at layer 4 of the Open Systems Interconnection (OSI) model. It's the single point of contact for clients. Load balancer distributes inbound flows that arrive at the load balancer's front end to backend pool instances. These flows are according to configured load-balancing rules and health probes. The backend pool instances can be Azure Virtual Machines or instances in a virtual machine scale set.

A **[public load balancer](https://docs.microsoft.com/en-us/azure/load-balancer/components#frontend-ip-configurations)** can provide outbound connections for virtual machines (VMs) inside your virtual network. These connections are accomplished by translating their private IP addresses to public IP addresses. Public Load Balancers are used to load balance internet traffic to your VMs.

An **internal (or private) load balancer** is used where private IPs are needed at the frontend only. Internal load balancers are used to load balance traffic inside a virtual network. A load balancer frontend can be accessed from an on-premises network in a hybrid scenario.



9. Why use Azure Load Balancer?

With Azure Load Balancer, you can scale your applications and create highly available services. Load balancer supports both inbound and outbound scenarios. Load balancer provides low latency and high throughput, and scales up to millions of flows for all TCP and UDP applications.

Key scenarios that you can accomplish using Azure Standard Load Balancer include:

- Load balance **internal** and **external** traffic to Azure virtual machines.
- Increase availability by distributing resources **within** and **across** zones.
- Configure **outbound connectivity** for Azure virtual machines.
- Use **health probes** to monitor load-balanced resources.
- Employ **port forwarding** to access virtual machines in a virtual network by public IP address and port.
- Enable support for **load-balancing** of **IPv6**.
- Standard load balancer provides multi-dimensional metrics through Azure Monitor. These metrics can be filtered, grouped, and broken out for a given dimension. They provide current and historic insights into performance and health of your service. Insights for Azure Load Balancer offers a preconfigured dashboard with useful visualizations for these metrics. Resource Health is also supported. Review **Standard load balancer diagnostics** for more details.
- Load balance services on **multiple ports, multiple IP addresses, or both**.
- Move **internal** and **external** load balancer resources across Azure regions.
- Load balance TCP and UDP flow on all ports simultaneously using **HA ports**.

NOTE: Delete all storage accounts as soon as you finished this lab